LSTM and GRU

Why they came, their architecture, the equations, and stacked (deep) RNNs — step by step

Moin Mostakim

Outline

1. Why LSTM and GRU came
2. LSTM architecture
3. GRU architecture
4. Stacked (deep) RNNs
5. Summary

The problem with vanilla RNNs

- A plain RNN updates a hidden state each step:

$$\mathbf{h}_t = \tanh(W_x \mathbf{x}_t + W_h \mathbf{h}_{t-1} + b).$$

- Training uses **backpropagation through time**: gradients are multiplied by W_h (and $\tanh'$) once per time step.
- Over long sequences this product either shrinks to 0 (**vanishing gradient**) or blows up (**exploding gradient**).

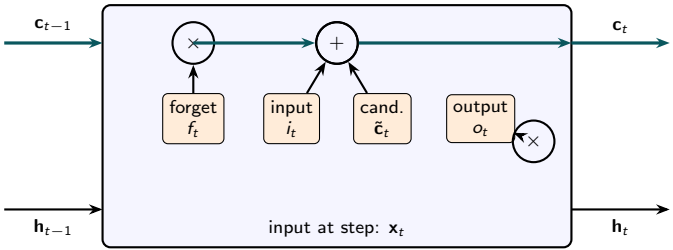
Consequence: vanilla RNNs cannot learn *long-range* dependencies — e.g. linking a word to context 50 steps earlier.

The core idea: a gated memory

- **LSTM** (Hochreiter & Schmidhuber, 1997) adds a separate **cell state** $\mathbf{c}_t$ that flows almost unchanged across time — a "*gradient highway*".
- **Gates** (learned sigmoid valves in $[0, 1]$) decide what to *keep*, *write*, and *read* from this memory.
- **GRU** (Cho et al., 2014) is a lighter redesign: it merges the gates and drops the separate cell state, keeping most of the benefit with fewer parameters.

	LSTM	**GRU**
States	$\mathbf{h}_t$ and $\mathbf{c}_t$	only $\mathbf{h}_t$
Gates	input, forget, output	update, reset
Parameters	more (4 weight sets)	fewer (3 weight sets)

LSTM cell — the two horizontal highways



Top line = cell state **c** (long-term memory). Bottom line = hidden state **h** (short-term / output). Gates modulate what crosses between them.

LSTM equations — step by step

At each time step t, given input $\mathbf{x}_t$ and previous $\mathbf{h}_{t-1}, \mathbf{c}_{t-1}$:

$$f_t = \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + b_f)$$ forget gate: what to *erase*

$$i_t = \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + b_i)$$ input gate: what to *write*

$$\tilde{\mathbf{c}}_t = \tanh(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + b_c)$$ candidate values

$$\mathbf{c}_t = f_t \odot \mathbf{c}_{t-1} + i_t \odot \tilde{\mathbf{c}}_t$$ update memory

$$o_t = \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + b_o)$$ output gate: what to *read*

$$\mathbf{h}_t = o_t \odot \tanh(\mathbf{c}_t)$$ new hidden state

σ = sigmoid (gate values in $[0, 1]$), $\odot$ = element-wise product. If $f_t \approx 1$ and $i_t \approx 0$, memory is carried unchanged — this is why gradients survive.

Reading each LSTM gate

Gate	Formula	Intuition
Forget f_t	$\sigma(\cdots)$	Scales old memory $\mathbf{c}_{t-1}$; 0 = forget, 1 = keep.
Input i_t	$\sigma(\cdots)$	How much of the new candidate to store.
Candidate $\tilde{\mathbf{c}}_t$	$\tanh(\cdots)$	Proposed new information to add.
Output o_t	$\sigma(\cdots)$	How much of the memory to expose as $\mathbf{h}_t$.

The additive update $\mathbf{c}_t = f_t \odot \mathbf{c}_{t-1} + i_t \odot \tilde{\mathbf{c}}_t$ (not a repeated matrix multiply) is the key fix for vanishing gradients.

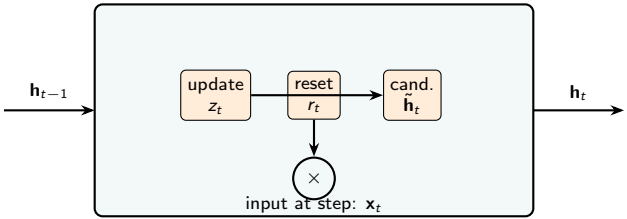
Worked example — one LSTM step (scalar)

Inputs: $x_t = 1.0,\ \mathbf{h}_{t-1} = 0.0,\ \mathbf{c}_{t-1} = 0.5.$ **Weights:** (W_f, U_f, b_f)=$(0.7, 0.6, 0.1)$, (W_i, U_i, b_i)=$(0.5, 0.4, 0.0)$, (W_c, U_c, b_c)=$(0.3, 0.2, 0.0)$, (W_o, U_o, b_o)=$(0.6, 0.5, 0.1)$.

$$
\begin{aligned}
f_t &= \sigma(0.7 \cdot 1 + 0.6 \cdot 0 + 0.1) = \sigma(0.8) = \mathbf{0.690} \\
i_t &= \sigma(0.5 \cdot 1 + 0.0) = \sigma(0.5) = \mathbf{0.622} \\
\tilde{\mathbf{c}}_t &= \tanh(0.3 \cdot 1 + 0.0) = \tanh(0.3) = \mathbf{0.291} \\
\mathbf{c}_t &= 0.690 \cdot 0.5 + 0.622 \cdot 0.291 = \mathbf{0.526} \\
o_t &= \sigma(0.6 \cdot 1 + 0.1) = \sigma(0.7) = \mathbf{0.668} \\
\mathbf{h}_t &= 0.668 \cdot \tanh(0.526) = 0.668 \cdot 0.483 = \mathbf{0.322}
\end{aligned}
$$

The memory moved $0.5 \to 0.526$ (mostly kept, a little written); the emitted hidden state is $\mathbf{h}_t = 0.322$.

GRU cell — a simpler gated unit



Only **one** state $\mathbf{h}_t$. The **update gate** z_t blends old and new; the **reset gate** r_t decides how much past to use when forming the candidate.

GRU equations — step by step

At each time step t, given input $\mathbf{x}_t$ and previous $\mathbf{h}_{t-1}$:

$$z_t = \sigma(W_z \mathbf{x}_t + U_z \mathbf{h}_{t-1} + b_z) \quad \text{update gate}$$

$$r_t = \sigma(W_r \mathbf{x}_t + U_r \mathbf{h}_{t-1} + b_r) \quad \text{reset gate}$$

$$\tilde{\mathbf{h}}_t = \tanh\big(W_h \mathbf{x}_t + U_h (r_t \odot \mathbf{h}_{t-1}) + b_h\big) \quad \text{candidate state}$$

$$\mathbf{h}_t = (1 - z_t) \odot \mathbf{h}_{t-1} + z_t \odot \tilde{\mathbf{h}}_t \quad \text{interpolate old/new}$$

If $z_t \approx 0$ the unit copies $\mathbf{h}_{t-1}$ (long memory); if $z_t \approx 1$ it fully refreshes. The reset gate r_t lets it drop irrelevant history when proposing $\tilde{\mathbf{h}}_t$.

LSTM vs. GRU — side by side

	LSTM	GRU
Memory	separate cell $\mathbf{c}_t$ + hidden $\mathbf{h}_t$	single hidden $\mathbf{h}_t$
Gates	forget, input, output (3)	update, reset (2)
Parameters	$\sim$33% more	fewer, faster to train
Long memory via	additive cell update	z_t interpolation
When to prefer	long/complex sequences, lots of data	smaller data, faster inference

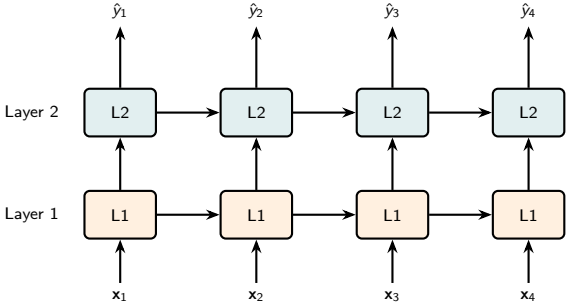
Both solve the vanishing-gradient problem; empirically their accuracy is often comparable, so GRU is a common lighter default.

Why stack recurrent layers

- A single recurrent layer learns one level of temporal features.
- **Stacking** feeds the hidden-state sequence of layer ℓ as the input sequence to layer $\ell+1$.
- Lower layers capture short, local patterns; higher layers capture longer, more abstract structure — the temporal analogue of deep CNNs.

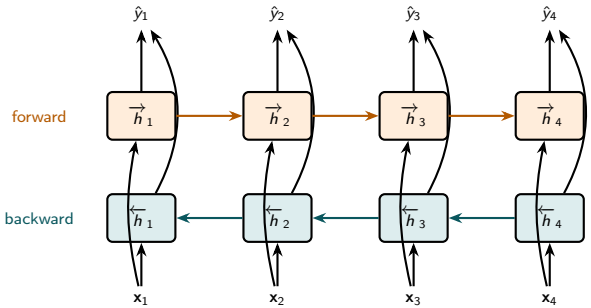
Rule: every layer returns its *full sequence* of hidden states to the next layer; only the *top* layer's output feeds the final prediction head.

Stacked LSTM/GRU — data flow



Horizontal arrows = recurrence (through time). Vertical arrows = depth (layer to layer). Each cell can be an LSTM or a GRU.

Bidirectional RNN — reading the sequence both ways



Two independent passes: a forward RNN reads $\mathbf{x}_1 \to \mathbf{x}_T$ and a backward RNN reads $\mathbf{x}_T \to \mathbf{x}_1$. The output uses both: $\hat{y}_t = g\big([\overrightarrow{h}_t; \overleftarrow{h}_t]\big)$, so each step sees *past and future* context.

When to use bidirectional

Good fit (whole sequence available)	**Not usable (must be causal / streaming)**
Text classification, NER, POS tagging	Real-time speech transcription
Offline speech recognition	Language generation / next-token prediction
Protein / DNA sequence labeling	Live time-series forecasting

Bidirectional doubles parameters and needs the *full* input before producing outputs — fine for analysis, impossible when the future is not yet known.

Stacking — step by step

For L stacked layers, at time t:

1. Layer 1 input is the data: $\mathbf{h}_t^{(0)} = \mathbf{x}_t$.
2. Each layer ℓ runs its own LSTM/GRU update:

$$\mathbf{h}_t^{(\ell)} = \mathrm{Cell}^{(\ell)}\big(\mathbf{h}_t^{(\ell-1)},\ \mathbf{h}_{t-1}^{(\ell)}\big).$$

3. The input to layer ℓ is the *hidden sequence* of layer $\ell - 1$.
4. The top layer's states $\mathbf{h}_t^{(L)}$ go to the output head (classifier / regressor / decoder).

Practical tips: 2–4 layers is typical; add *dropout between layers* to regularize; use *residual connections* for very deep stacks; a *bidirectional* wrapper runs one stack forward and one backward.

Stacked LSTM in code (Keras / PyTorch)

Keras:

```
model = Sequential([
    LSTM(128, return_sequences=True,  input_shape=(T, d)),  # layer 1
    Dropout(0.2),
    LSTM(64,  return_sequences=False),                      # layer 2 (top)
    Dense(num_classes, activation='softmax')
])
```

PyTorch (one call, `num_layers` stacks it):

```
self.rnn = nn.LSTM(input_size=d, hidden_size=128,
                   num_layers=3, batch_first=True, dropout=0.2)
# GRU: nn.GRU(... same arguments ...)
```

`return_sequences=True` on all but the last layer is the "pass the full sequence up" rule.

Summary

- **Why:** vanilla RNNs suffer vanishing/exploding gradients and cannot learn long-range dependencies.
- **LSTM:** cell state $\mathbf{c}_t$ + forget/input/output gates; additive memory update preserves gradients.
- **GRU:** single state $\mathbf{h}_t$ with update/reset gates — lighter, fewer parameters, similar accuracy.
- **Stacking:** feed each layer's hidden sequence into the next; lower layers = local features, higher layers = abstract temporal structure.
- **Use:** 2–4 layers with inter-layer dropout; optionally bidirectional or residual.